

Chuyên đề: Đánh Giá Độ Phức Tạp Thuật Toán & Tối Ưu Hóa Hiệu Năng (Computational Complexity & Performance Optimization)

(Asymptotic Notations, Constraint-to-Algorithm Mapping, Amortized Analysis, Master Theorem & Cache Locality)

Tác giả: Duc-Minh Vu | **Đơn vị:** SLSCM Lab — Faculty of Data Science and Artificial Intelligence (FDA), Đại học Kinh tế Quốc dân (NEU)

Soạn thảo: Được hỗ trợ và soạn thảo bởi Agentic AI tool

Bản quyền © 2026 Duc-Minh Vu. Mọi quyền được bảo lưu. Tài liệu đào tạo và bồi dưỡng sinh viên Olympic Tin học / ICPC.

Sơ đồ Tư duy Toàn cảnh (Mindmap)



1. Cơ Sở Toán Học: Ký Hiệu Tiệm Cận & Quy Tắc 1 Giây

1.1. Các Ký Hiệu Tiệm Cận (Asymptotic Notations)

Khi đánh giá hiệu năng của một thuật toán, ta quan tâm đến **tốc độ tăng trưởng của thời gian chạy hoặc dung lượng bộ nhớ** khi kích thước đầu vào N tiến tới vô cùng ($N \rightarrow \infty$).

1. Ký hiệu O lớn (*Big-O* — Cận trên tiệm cận):

$$f(N) = O(g(N)) \iff \exists c > 0, N_0 > 0 \text{ sao cho } 0 \leq f(N) \leq c \cdot g(N) \quad \forall N \geq N_0$$

Ý nghĩa trong CP: Đại diện cho **thời gian chạy trong trường hợp xấu nhất (Worst-case)**. Đây là thước đo tiêu chuẩn để đảm bảo code không bị Time Limit Exceeded (TLE).

2. Ký hiệu Ω lớn (*Big-Ω* — Cận dưới tiệm cận):

$$f(N) = \Omega(g(N)) \iff \exists c > 0, N_0 > 0 \text{ sao cho } 0 \leq c \cdot g(N) \leq f(N) \quad \forall N \geq N_0$$

Ý nghĩa trong CP: Đại diện cho thời gian chạy trong trường hợp tốt nhất (Best-case) hoặc cận dưới lý thuyết của bài toán (ví dụ: Sắp xếp dựa trên so sánh cần $\Omega(N \log N)$).

3. Ký hiệu Θ lớn (*Big-Θ* — Cận chặt tiệm cận):

$$f(N) = \Theta(g(N)) \iff f(N) = O(g(N)) \quad \text{và} \quad f(N) = \Omega(g(N))$$

Ý nghĩa trong CP: Thuật toán chạy đúng với hàm tăng trưởng $g(N)$ trong mọi trường hợp (ví dụ: Merge Sort luôn là $\Theta(N \log N)$).

1.2. Quy Tắc 1 Giây (10^8 Thao Tác Cơ Bản)

Trong các hệ thống chấm bài trực tuyến chuẩn mực (Codeforces, AtCoder, VNOI, CSES, ICPC):

- Bộ vi xử lý hiện đại (Intel Core i7/AMD Ryzen/Xeon) có xung nhịp khoảng 2.5 . . . 4.0 GHz.
- Một chu kỳ xung nhịp có thể thực hiện từ 1 đến vài chỉ lệnh hợp ngữ (Instructions Per Cycle - IPC).
- Tuy nhiên, trong ngôn ngữ cấp cao (C++), một "thao tác cơ bản" (gán, so sánh, truy cập mảng) tương ứng với 2–5 chỉ lệnh máy.

🚩 Quan trọng:

****Quy tắc bỏ túi kinh nghiệm (Rule of Thumb):****

Trong giới hạn thời gian ****1.0 giây****, một chương trình C++ được tối ưu hóa hợp lý (`-O2` / -O3``) có thể thực hiện an toàn:

$$N_{\text{ops}} \approx 10^8 \text{ thao tác cơ bản}$$

- ****Ngưỡng an toàn ($< 0.5\text{s}$):**** $\leq 5 \times 10^7$ thao tác.
- ****Ngưỡng chậm trễ ($0.8\text{s} - 1.0\text{s}$):**** $\approx 10^8$ thao tác (cần tránh phép chia, modulo, hoặc cấp phát động).
- ****Nguy cơ TLE ($> 1.5\text{s}$):**** $\geq 2 \times 10^8$ thao tác.

2. Bảng Tra Cứu "Nhìn Ràng Buộc Đoán Thuật Toán" (Constraint-to-Algorithm Cheatsheet)

Trong phòng thi ICPC / Olympic Tin học, bước đầu tiên khi đọc đề bài là **nhìn ngay vào ràng buộc dữ liệu** (Constraints). Kích thước của N sẽ tiết lộ chính xác lớp thuật toán mà tác giả đề bài nhắm tới:

Ràng buộc dữ liệu (N)	Độ phức tạp thời gian mục tiêu	Lớp thuật toán & Cấu trúc dữ liệu điển hình
$N \leq 10 \dots 11$	$O(N!)$ hoặc $O(N^2 \cdot 2^N)$	Quay lui hoán vị (<code>std::next_permutation</code>), Nhánh cận, TSP
$N \leq 20 \dots 22$	$O(2^N)$ hoặc $O(N \cdot 2^N)$	Quy hoạch động trạng thái (Bitmask DP), Duyệt tập con (Submask enumeration)
$N \leq 40 \dots 45$	$O(2^{N/2} \cdot \frac{N}{2})$	Gặp nhau ở giữa (Meet-in-the-middle), Chia đôi tập hợp
$N \leq 100 \dots 400$	$O(N^4)$ hoặc $O(N^3)$	Floyd-Warshall, Khử Gauss trên $\mathbb{R}$ hoặc $\mathbb{F}_2$, Nhân ma trận, DP 3 chiều

Ràng buộc dữ liệu (N)	Độ phức tạp thời gian mục tiêu	Lớp thuật toán & Cấu trúc dữ liệu điển hình
$N \leq 1000 \dots 2000$	$O(N^2)$	Quy hoạch động 2D, Dijkstra mảng, Duyệt mọi cặp đỉnh/đoạn con, CHT $O(N^2)$
$N \leq 5000$	$O(N^2)$ cực nhanh	N^2 với hằng số nhỏ (Duyệt mảng phẳng, tối ưu bằng <code>std::bitset</code> $\frac{N^2}{64}$)
$N \leq 10^5 \dots 2 \times 10^5$	$O(N\sqrt{N})$	Thuật toán Mo, Chia căn (Sqrt Decomposition), Chia khối truy vấn
$N \leq 2 \times 10^5 \dots 5 \times 10^5$	$O(N \log N)$ hoặc $O(N \log^2 N)$	" Vùng đất vàng " của CP: Segment Tree, Fenwick Tree, Sparse Table, DSU, Sắp xếp, Chặt nhị phân, Convex Hull Trick, Sweep-line
$N \leq 10^6 \dots 2 \times 10^6$	$O(N)$ hoặc $O(N \log N)$ nhẹ	Two Pointers, Monotonic Stack, Prefix Sum, BFS/DFS, Sàng Eratosthenes, KMP, Hashing
$N \leq 10^7 \dots 10^8$	$O(N)$ cực sạch	Sàng tuyến tính $O(N)$, Mảng 1D, Fast I/O, Vòng lặp đơn giản không rẽ nhánh
$N \leq 10^9 \dots 10^{18}$	$O(\sqrt{N})$, $O(\log N)$, $O(1)$	Kiểm tra nguyên tố $O(\sqrt{N})$, Sàng Đỗ Hồng Cường $O(N^{2/3})$, Lũy thừa ma trận $O(K^3 \log N)$, Euclid gcd, Công thức đóng $O(1)$

3. Phân Tích Khấu Hao (Amortized Analysis)

Một sai lầm kinh điển của người mới học CP là nhìn thấy **hai vòng lặp lồng nhau** (vòng lặp `while` nằm trong `for`) liền kết luận ngay độ phức tạp là $O(N^2)$. Trên thực tế, rất nhiều thuật toán có cấu trúc này lại chạy trong $O(N)$ **tuyến tính** nhờ phân tích khấu hao!

3.1. Ba Phương Pháp Phân Tích Khấu Hao

1. Phương pháp Tổng gộp (Aggregate Analysis):

Chứng minh rằng tổng thời gian của chuỗi N thao tác trong trường hợp xấu nhất bị chặn bởi $T(N) \leq c \cdot N$. Khi đó chi phí trung bình (khấu hao) của mỗi thao tác là:

$$\text{Amortized Cost} = \frac{T(N)}{N} = O(1)$$

2. Phương pháp Kế toán (Accounting Method):

Gán cho mỗi thao tác một "khoản phí" ảo. Thao tác đơn giản được tính thừa tiền để dành vào "tài khoản ngân hàng", dùng để chi trả cho các thao tác phức tạp tốn kém diễn ra sau đó mà không bao giờ bị âm quỹ.

3. Phương pháp Thế năng (Potential Method):

Định nghĩa một hàm thế năng $\Phi(D) \geq 0$ phản ánh "mức độ lộn xộn" của cấu trúc dữ liệu.

Chi phí khấu hao: $a_i = c_i + \Phi(D_i) - \Phi(D_{i-1})$.

Tổng chi phí thực tế: $\sum c_i = \sum a_i - \Phi(D_N) + \Phi(D_0) \leq \sum a_i$.

3.2. Case Studies Phân Tích Khấu Hao Thực Chiến

1. Ngăn Xếp Đơn Điều (Monotonic Stack) & Hai Con Trỏ (Two Pointers)

Xét vòng lặp tìm phần tử nhỏ hơn gần nhất:

```
for (int i = 0; i < n; ++i) {
    while (!st.empty() && st.top() >= a[i]) {
        st.pop();
    }
    st.push(a[i]);
}
```

- **Phân tích sai:** Vòng `for` chạy N lần, vòng `while` bên trong chạy tối đa N lần $\implies O(N^2)$ (Sai!).
- **Phân tích khấu hao chuẩn:**
 - Mỗi phần tử được `push` vào ngăn xếp **đúng 1 lần**.
 - Mỗi phần tử bị `pop` ra khỏi ngăn xếp **tối đa 1 lần**.
 - Tổng số thao tác `pop` trên toàn bộ thời gian chạy của thuật toán không thể vượt quá tổng số thao tác `push` $\implies$ Tổng số lần thực thi thân vòng lặp `while` $\leq N$.
- Tổng thời gian: $O(N)$!

2. Kỹ thuật Gộp Tập Hợp Nhỏ Vào Lớn (Small-to-Large Merging / DSU on Tree)

Khi gộp hai tập hợp có kích thước A và B ($|A| \leq |B|$):

- Ta duyệt từng phần tử của tập nhỏ A và chèn vào tập lớn B .
- Mỗi lần một phần tử bị duyệt và di chuyển, kích thước của tập hợp chứa nó **tăng ít nhất gấp đôi**:

$$\text{Size}_{\text{new}} \geq 2 \cdot \text{Size}_{\text{old}}$$

- Kích thước tối đa của tập hợp không vượt quá $N \implies$ Một phần tử chỉ có thể bị di chuyển tối đa $\lfloor \log_2 N \rfloor$ lần trong toàn bộ quá trình chạy của chương trình!
- Có N phần tử $\implies$ Tổng thời gian gộp của toàn bộ thuật toán là $O(N \log N)$ (thay vì $O(N^2)$)!

4. Đệ Quy & Định Lý Thợ (Master Theorem)

Trong các thuật toán Chia để trị (Divide and Conquer), thời gian thực thi $T(N)$ thỏa mãn phương trình truy hồi:

$$T(N) = a \cdot T\left(\frac{N}{b}\right) + f(N) = a \cdot T\left(\frac{N}{b}\right) + O(N^d)$$

- $a \geq 1$: Số bài toán con được sinh ra ở mỗi bước chia.
- $b > 1$: Tỷ lệ thu nhỏ kích thước bài toán.
- $f(N) = O(N^d)$: Chi phí chia nhỏ bài toán và gộp kết quả.

4.1. Ba Trường Hợp Cốt Lõi của Master Theorem

Đặt giá trị tới hạn: $c_{\text{crit}} = \log_b a$.

Trường hợp	So sánh d và $\log_b a$	Ý nghĩa bản chất	Độ phức tạp tổng quát $T(N)$	Thuật toán điển hình
Trường hợp 1	$d < \log_b a$	Chi phí tại các nút lá chiếm ưu thế tuyệt đối	$\Theta(N^{\log_b a})$	Nhân ma trận Strassen ($a = 7, b = 2 \implies O(N^{2.81})$), Karatsuba ($a = 3, b = 2 \implies O(N^{1.58})$)
Trường hợp 2	$d = \log_b a$	Chi phí phân bố đều đặn trên tất cả các tầng của cây đệ quy	$\Theta(N^d \log N)$	Merge Sort ($a = 2, b = 2, d = 1 \implies O(N \log N)$), Xây dựng Segment Tree
Trường hợp 3	$d > \log_b a$	Chi phí tại nút gốc chiếm ưu thế tuyệt đối	$\Theta(N^d)$	Thuật toán tìm trung vị QuickSelect ($a = 1, b = 2, d = 1 \implies O(N)$)

4.2. Chuỗi Điều Hòa (Harmonic Series) Trong Lý Thuyết Số

Xét thuật toán đếm số ước hoặc sàng ước số:

```
for (int i = 1; i <= n; ++i) {
    for (int j = i; j <= n; j += i) {
        // Xử lý bội số j của i
    }
}
```

Số lần thực thi của vòng lặp bên trong là:

$$S = \sum_{i=1}^N \left\lfloor \frac{N}{i} \right\rfloor \approx N \sum_{i=1}^N \frac{1}{i} = N \cdot H_N$$

Theo giải tích toán học, tổng điều hòa $H_N = 1 + \frac{1}{2} + \frac{1}{3} + \dots + \frac{1}{N} = \ln N + \gamma + O\left(\frac{1}{N}\right)$ (với $\gamma \approx 0.5772$ là hằng số Euler-Mascheroni).

Do đó:

$$S = O(N \log N)$$

(Với $N = 10^6$, $N \ln N \approx 1.38 \times 10^7$ thao tác, chạy trong chưa đầy 0.05 giây!)

5. Phần Cứng, Bộ Nhớ & Chi Phí Ẩn (Hidden Constants & Hardware Optimization)

Hai đoạn code có cùng độ phức tạp tiệm cận $O(N)$ có thể chênh lệch thời gian chạy từ **3 đến 10 lần** do sự tương tác với kiến trúc phần cứng máy tính hiện đại.

5.1. Bộ Nhớ Đệm & Tính Cục Bộ (Cache Locality)

CPU không nạp từng byte riêng lẻ từ RAM mà nạp theo từng khối liên tục gọi là **Cache Line (thường là 64 bytes)** vào bộ nhớ đệm L1/L2/L3.

Truy cập liên tục (Row-Major): [A[0][0] -> A[0][1] -> A[0][2] -> ...] ==> 1 Cache Miss mỗi 16
Truy cập nhảy cóc (Column-Major): [A[0][0] -> A[1][0] -> A[2][0] -> ...] ==> Cache Miss liên tục

```
// THÍ NGHIỆM THỰC TẾ: Ma trận 4096 x 4096
// Cách 1: Tối ưu Cache (Duyệt hàng trước, cột sau) -> Chạy ~ 15 ms
for (int i = 0; i < N; ++i)
    for (int j = 0; j < N; ++j) sum += a[i][j];

// Cách 2: Phá vỡ Cache (Duyệt cột trước, hàng sau) -> Chạy ~ 180 ms (Chậm hơn 12 lần!)
for (int j = 0; j < N; ++j)
    for (int i = 0; i < N; ++i) sum += a[i][j];
```

5.2. Bảng Chi Phí Lệnh CPU (Instruction Latency)

Phép toán C++	Số chu kỳ CPU (Clock Cycles)	Lời khuyên tối ưu trong CP
Phép toán Bit (& , \ , ^ , ~ , << , >>)	1 cycle	Rất nhanh, ưu tiên sử dụng thay thế phép nhân chia lũy thừa 2.
Cộng, Trừ (+ , -)	1 cycle	Thao tác cơ bản nhanh nhất của ALU.
Nhân số nguyên (*)	3 – 4 cycles	Đã được phần cứng hỗ trợ pipeline rất tốt.
Chia và Modulo (/ , %)	15 – 25 cycles	Rất chậm! Hạn chế tối đa dùng % trong các vòng lặp 10^8 .
Cấp phát động (new , malloc , vector.resize)	** hàng trăm cycles**	Cực chậm vì phải gọi System Call của Hệ điều hành. Nên cấp phát mảng tĩnh trước.

💡 Mẹo / Tối ưu:

Tối ưu Modulo trong Quy hoạch động:

Thay vì viết `sum = (sum + val) % MOD;` trong vòng lặp chạy 10^8 lần, hãy viết:

```
```cpp
```

```
sum += val;
if (sum >= MOD) sum -= MOD;
...
```

Phép trừ đơn giản chỉ tốn 1 cycle, nhanh gấp **\*\*15 lần\*\*** so với phép toán `%`!

### 5.3. Tính Toán Dung Lượng Bộ Nhớ (Memory Footprint)

Trong các đề thi, giới hạn bộ nhớ thường là **256 MB** hoặc **512 MB**.

$$\text{Dung lượng (Bytes)} = \text{Số phần tử} \times \text{Kích thước kiểu dữ liệu}$$

- `bool`, `char`: 1 Byte
- `int`, `float`: 4 Bytes
- `long long`, `double`, con trỏ 64-bit: 8 Bytes
- `__int128`: 16 Bytes

#### Bảng chuyển đổi nhanh an toàn:

- Mảng `int a[10000000]` ( $10^7$  phần tử) tốn  $10^7 \times 4 \text{ B} \approx 40 \text{ MB}$ .
- Mảng `long long a[10000000]` ( $10^7$  phần tử) tốn  $10^7 \times 8 \text{ B} \approx 80 \text{ MB}$ .
- Ma trận 2D `int a[5000][5000]` tốn  $2.5 \times 10^7 \times 4 \text{ B} \approx 100 \text{ MB} \implies$  An toàn cho 256MB.
- Ma trận 2D `int a[10000][10000]` tốn  $10^8 \times 4 \text{ B} \approx 400 \text{ MB} \implies$  **Tràn bộ nhớ (MLE)!** Bắt buộc phải tối ưu bằng mảng con lăn (Rolling Array) hoặc `std::vector<bool>`.

## 6. Chạm Bẫy Phổ Biến Dẫn Đến TLE / MLE / RE

### 1. Truyền Tham Trị Thay Vì Tham Chiếu Trong Hàm:

```
```cpp
// SAI LẦM TAI HẠI: Mỗi lần gọi đệ quy sẽ copy toàn bộ vector tốn O(N) -> O(N^2) TLE!
void dfs(int u, vector adj) { ... }
```

// CHUẨN XÁC: Truyền tham chiếu hằng tốn $O(1)$

```
void dfs(int u, const vector& adj) { ... }
```

2. **Tràn Ngăn Xếp Đệ Quy (Stack Overflow):** Trong các bài toán duyệt cây hoặc đồ thị có dạng suy biến thành đường thẳng ($N = 2 \times 10^5$), độ sâu đệ quy đạt 2×10^5 . Kích thước ngăn xếp mặc định trên một số hệ thống chỉ là 8MB $\implies$ Bị **Segmentation Fault (Runtime Error)**. Giải pháp: Mở rộng stack size trên Linux (`ulimit -s`) hoặc dùng DFS khử đệ quy bằng `std::vector`.

3. **Va Chạm Bảng Băm $O(N^2)$ Trên `std::unordered_map`:** Hàm băm mặc định của GCC trên số nguyên là hàm đồng nhất ($\text{hash}(x) = x$). Trên Codeforces, hacker có thể dễ dàng tạo ra bộ test dữ liệu các số nguyên có cùng số dư modulo $2^{62}-1$, đẩy độ phức tạp của `unordered_map` từ $O(1)$ lên $O(N)$ mỗi thao tác $\implies$ Toàn bộ code bị TLE $O(N^2)$! ***Giải pháp:** Luôn sử dụng bộ băm chống hack `custom_hash` (dựa trên thuật toán SplitMix64).

7. Tài liệu Tham khảo & Đọc thêm

1. **VNOI Wiki**:
 2. [Độ phức tạp tính toán](#) — Đánh giá thời gian, bộ nhớ và tiệm cận.
 3. **Introduction to Algorithms (CLRS 4th Edition)**:
 4. *Chapter 3: Asymptotic Notation & Chapter 17: Amortized Analysis.*
 5. **Codeforces Community**:
 6. [Asymptotic Complexity and Hardware Limits in CP](#) — Quy tắc 10^8 thao tác/giây và bộ nhớ đệm CPU L1/L2.
 7. **Competitive Programmer's Handbook** — *Antti Laaksonen*:
 8. [Chapter 3: Efficiency & Optimizations](#) — Hằng số thực thi và tối ưu hóa bitwise.
-



Competitive Programming Handbook

SLSCM Lab — Faculty of Data Science and Artificial Intelligence (FDA) — National Economics University (NEU)

Tài liệu được soạn thảo và tối ưu bởi Agentic AI tool

© 2026 Duc-Minh Vu. Toàn bộ bản quyền được bảo lưu.